Fabian Alexander Feith
Inalbek Akiev

Fakultät 10
Robotik
Prof. Dr. Algorri

Zur Realisierung des Projekts werden hauptsächlich drei Nodes verwendet: camera_publisher.py, correction_vector.py und sketch_Servo360_microROS.ino. Zusätzlich verwenden wir den bereitgestellten Micro-ROS Agent. Ziel des Systems ist ein autonomes Fahrzeug, das einer Spur aus zwei Linien folgt. Im ersten Schritt steht eine robuste und stabile Fahrweise im Vordergrund. Erst danach soll das System durch gezieltes Feintuning hinsichtlich der Geschwindigkeit optimiert werden.

Die Architektur trennt die rechenintensive Bildverarbeitung und Entscheidungslogik auf dem Raspberry Pi von der zeitkritischen Ansteuerung der Servos auf dem Arduino. Dadurch kann der Raspberry Pi die Kamerabilder aufnehmen, verarbeiten und daraus einen Steuerwert berechnen, während der Arduino diesen Wert in Servosignale umrechnet. Die Kommunikation zwischen beiden Systemen erfolgt über das ROS2-Topic micro_ros_fabinal_subscriber, auf dem ein Int32-Steuerwert übertragen wird.

```
publisher = self.create_publisher(Int32, 'micro_ros_fabinal_subscriber', 10)
```

Diese Aufteilung ist für das Gesamtsystem entscheidend, weil sie Bildverarbeitung, Entscheidungslogik und Servosteuerung klar voneinander trennt und damit sowohl die Stabilität als auch die Reaktionsfähigkeit verbessert.

Kamerasystem und Bildaufnahme

Die Node camera_publisher.py initialisiert die Kamera über OpenCV und veröffentlicht die aufgenommenen Bilder kontinuierlich als ROS2-Nachrichten vom Typ sensor_msgs/Image. Die Bildrate beträgt etwa 30 Hz, was durch einen Timer realisiert wird. Gleichzeitig ist die Kamera auf eine feste Auflösung von 320 × 240 Pixeln eingestellt.

```
self.publisher_ = self.create_publisher(Image, 'camera_image', 10)
timer_period = 0.016
self.timer = self.create_timer(timer_period, self.timer_callback)
self.cap = cv2.VideoCapture(0)
self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
```

Die reduzierte Auflösung ist eine zentrale Stellschraube, da sie die Anzahl der zu verarbeitenden Pixel deutlich verringert. Dadurch sinkt der Rechenaufwand für die nachfolgende Grauwertumwandlung, Threshold-Bildung, Konturenerkennung und Linienapproximation. Das System kann so schneller auf Änderungen in der Spur reagieren, was vor allem für höhere Fahrgeschwindigkeiten wichtig ist.

Die Kameraaufnahme wird nur dann veröffentlicht, wenn ein gültiger Frame gelesen wurde.

```
ret, frame = self.cap.read()
if ret:
    msg = self.br.cv2_to_imgmsg(frame, encoding="bgr8")
    self.publisher_.publish(msg)
else:
    self.get_logger().error('Error in capturing image')
```

Damit ist sichergestellt, dass nur gültige Bilddaten in die weitere Verarbeitung eingehen.

Bildverarbeitung und Steuerwertberechnung

Die Node correction_vector.py übernimmt die gesamte Bildverarbeitung. Zunächst wird das eingehende Kamerabild in Graustufen umgewandelt und anschließend mit einem festen Schwellwert binarisiert. Der im Code verwendete Schwellwert beträgt THRESHOLD = 190.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
_, mask = cv2.threshold(gray, THRESHOLD, 255, cv2.THRESH_BINARY)
```

Anschließend wird eine Region of Interest definiert, die nur den unteren Bildbereich auswertet. Der Startpunkt dieser ROI liegt bei ROI_START = 0.2, sodass die untersten 80 % des Bildes verwendet werden. Dieser Bereich ist besonders relevant, weil er die fahrzeugnahe Spurinformation enthält und damit für die Regelung am wichtigsten ist.

```
roi_y = int(height * ROI_START)
roi = mask[roi_y:, :]
```

Innerhalb dieser ROI werden Konturen mit cv2.findContours extrahiert und anhand ihrer Fläche gefiltert. Nur Konturen mit mindestens MIN_CONTOUR_AREA = 100 werden weiterverarbeitet.

```
contours, _ = cv2.findContours(roi, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

if cv2.contourArea(cnt) < MIN_CONTOUR_AREA:
    continue
```

Für jede relevante Kontur wird anschließend mit cv2.fitLine eine Gerade gefittet. Aus dieser Geraden werden zwei Größen berechnet. Der Winkel der Linie zur Senkrechten und die x-Position der Linie am unteren Rand des ROI. Diese Position ist robuster als eine Bounding-Box-Mitte, weil sie näher am Fahrzeug liegt und damit besser für die Seitenzuordnung geeignet ist.

```
[vx, vy, x0, y0] = cv2.fitLine(contour, cv2.DIST_L2, 0, 0.01, 0.01).flatten()

if abs(vy) < 1e-6:
    angle_deg = 90.0
```

```
    cx = float(x0)
else:
    angle_deg = float(np.degrees(np.arctan2(vx, vy)))
    t = (roi_height - y0) / vy
    cx = float(x0 + t * vx)
```

Die Winkelwerte werden danach in den Bereich von −90° bis +90° normalisiert.

```
if angle_deg > 90:
    angle_deg -= 180
elif angle_deg < -90:
    angle_deg += 180
```

Diese Kombination aus Winkel und unterer x-Position bildet die Grundlage für die gesamte Spurzuordnung.

Zuordnung der Linien und Steuerlogik

Die erkannten Linien werden in linker und rechter Fahrspur zugeordnet. Dafür verwendet der Code ein Memory-System mit den zuletzt bekannten Positionen last_left_cx und last_right_cx. Neue Kandidaten werden nur dann einer Seite zugewiesen, wenn sie sich nicht zu stark von der zuletzt bekannten Position unterscheiden. Der maximale erlaubte Sprung beträgt MAX_JUMP_PX = 150.

```
if abs(c[1] - self.last_left_cx) <= MAX_JUMP_PX:
    left = c
```

Damit wird verhindert, dass einzelne falsch erkannte Konturen die Spurzuordnung destabilisieren. Wenn eine Seite über mehrere Frames nicht mehr erkannt wird, wird ihr Memory nach MEMORY_TIMEOUT_FRAMES = 10 zurückgesetzt.

Zusätzlich prüft der Code, ob die beiden Linien überhaupt als gültiges Spurpaar gelten. Dafür ist der Mindestabstand zwischen linker und rechter Linie mit MIN_LANE_WIDTH_PX = 300 festgelegt. Liegen zwei Linien zu nah beieinander, wird eine davon verworfen, damit keine falsche Spurinterpretation entsteht.

Die eigentliche Steuerung basiert, wenn beide Linien sichtbar sind, auf der Differenz ihrer Winkelbeträge. Daraus wird ein Fehler berechnet, der anschließend auf den maximal zulässigen Steuerwert begrenzt wird.

```
error = abs(left[0]) - abs(right[0])
steering = int(np.clip(-error * ANGLE_SCALE, -MAX_STEERING, MAX_STEERING))
```

Die Skalierung erfolgt mit ANGLE_SCALE = 1, der maximale Steuerwert liegt bei MAX_STEERING = 50. Ist nur eine Linie sichtbar, verwendet der Code heuristische Regeln, um trotzdem eine Fahrtrichtung abzuleiten. In diesem Fall wird bei einer linken

Linie standardmäßig ein positiver Steuerwert ausgegeben, bei einer rechten Linie ein negativer. So bleibt das Fahrzeug auch bei unvollständiger Linienerkennung handlungsfähig.

Corner-Erkennung und Sonderfälle

Eine wichtige Erweiterung ist die Corner-Erkennung. Sie greift, wenn eine einzelne Linie einen großen Winkel aufweist. Der Grenzwert dafür liegt bei SHARP_CORNER_ANGLE_DEG = 50. Zusätzlich wird nicht sofort reagiert, sondern die Situation muss über CORNER_DELAY_S = 0.2 Sekunden stabil anliegen. Erst dann wird ein starker Lenkbefehl ausgegeben.

```
SHARP_CORNER_ANGLE_DEG = 50
CORNER_DELAY_S = 0.2
```

Die Entscheidung berücksichtigt dabei nicht nur den Winkel, sondern auch die geometrische Lage der Linie. Dafür werden zwei Hilfsprüfungen verwendet: crosses_to_center und exits_same_side. Diese helfen zu unterscheiden, ob eine Linie in Richtung Bildmitte führt oder eine scharfe Kurve andeutet.

```
def crosses_to_center(line):
    angle_deg, cx = line
    if abs(angle_deg) < SHARP_CORNER_ANGLE_DEG:
        return False
    return (cx > mid and angle_deg < 0) or (cx < mid and angle_deg > 0)
```

Wenn keine gültigen Seitenlinien erkannt werden, prüft das System zusätzlich auf horizontale Strukturen. Liegt eine nahezu horizontale Linie vor, wird sie als Sonderfall interpretiert und das Fahrzeug entsprechend in die freie Richtung gelenkt. Diese Logik dient als Fallback für schwierige Bildsituationen.

Arduino und Servosteuerung

Der Arduino empfängt den vom Raspberry Pi berechneten Steuerwert über Micro-ROS. Im Subscription-Callback wird der empfangene Int32-Wert direkt in der globalen Variable steering gespeichert.

```
void subscription_callback(const void * msgin)
{
  const std_msgs__msg::Int32 * msg = (const std_msgs__msg::Int32 *)msgin;
  steering = msg->data;
}
```

Die eigentliche Fahrlogik befindet sich in driveRaceTrack(int steering). Dort werden zwei Betriebsarten unterschieden. Im Normalmodus wird eine differentielle Korrektur angewendet. Die Basissignale der Servos sind mit baseRight = 1415 und baseLeft = 1610 festgelegt. Je nach Vorzeichen und Größe des Steuerwerts werden daraus Korrekturwerte berechnet und anschließend mit writeMicroseconds an die Servos ausgegeben.

```
int baseRight = 1415;
int baseLeft = 1610;

servoRight.writeMicroseconds(baseRight + correctionRight);
servoLeft.writeMicroseconds(baseLeft + correctionLeft);
```

Im Sharp-Corner-Modus wird die normale Regelung übersteuert. Sobald der Betrag des Steuerwerts den Schwellwert SPOT_TURN_THRESHOLD = 90 überschreitet, wird ein spezielles Fahrverhalten aktiviert. Dabei wird das kurveninnere Rad stark reduziert und das äußere Rad mit einem festen Restwert weiter angesteuert. Das Fahrzeug kann dadurch sehr enge Kurven oder starke Richtungswechsel fahren.

```
if (abs(steering) >= SPOT_TURN_THRESHOLD) {
   servoRight.attach(pinControlRight);
   servoLeft.attach(pinControlLeft);

   int factor = 2.2;
   if (steering > 0) {
     servoLeft.writeMicroseconds(baseLeft + factor * (steering - 20));
     servoRight.writeMicroseconds(baseRight + 50);
   } else {
     servoRight.writeMicroseconds(baseRight + factor * (steering + 20));
     servoLeft.writeMicroseconds(baseLeft - 50);
   }
   return;
}
```

Gesamtsystem

Insgesamt bildet das Projekt einen vollständigen Vision-to-Control-Loop. Die Kamera erfasst die Umgebung, die Bildverarbeitung extrahiert daraus die relevanten Spurinformationen, und aus diesen Informationen wird ein Steuerwert erzeugt, der auf dem Arduino in konkrete Servosignale umgesetzt wird.

Die wichtigsten Stellschrauben im System sind dabei die Bildauflösung, der Threshold-Wert, die ROI-Grenze, das Memory-System zur Seitenzuordnung, die Corner-Erkennung mit Zeitverzögerung sowie die Servo-Basiswerte und Schwellenwerte auf dem Arduino. Genau diese Parameter bestimmen maßgeblich, wie stabil, robust und

schnell das Fahrzeug auf Änderungen der Spur reagiert und wurden während der Tests soweit optimiert, dass der Roboter die Strecke in beide Richtungen robust und unter einer Minute fährt.